Koopman Operator Learning for Nonlinear Dynamics

DMD, EDMD and Deep Koopman on the Nonlinear Pendulum and the Lorenz Attractor

October 2026

Abstract

This report documents a Python library that applies Koopman operator theory to nonlinear dynamical systems. It contains custom ODE solvers for data generation, exact Dynamic Mode Decomposition (DMD) computed through the SVD, Extended DMD (EDMD) with physics-informed observables, and a PyTorch Deep Koopman autoencoder that learns a latent space with linear dynamics, together with visualization tools and a test suite. The three methods are compared on a damped nonlinear pendulum (a system with a stable equilibrium and amplitude-dependent oscillation frequency) and on the chaotic Lorenz attractor, always by *predicting* from a single initial state on held-out data. Besides the results, the report explains how to read every figure that the code produces.

Contents

1 What was built **2**

2 Theory **2**

2.1 The Koopman operator 2

2.2 Exact DMD 3

2.3 Extended DMD 3

2.4 Deep Koopman autoencoder 3

3 Implementation notes **4**

4 Experimental protocol **4**

5 Results **5**

5.1 Pendulum 5

5.2 Lorenz attractor 7

5.3 Remaining figures 9

6 How to read each figure **14**

7 Limitations and honest caveats **15**

8 Reproducing the results **15**

1 What was built

The repository implements the following components. Each corresponds to a module of the `koopman` package.

Module	Content
`solvers.py`	Explicit Euler, classical RK4 and an adaptive Dormand–Prince 5(4) integrator, written from scratch.
`systems.py`	Damped nonlinear pendulum and Lorenz-63; trajectory and dataset generation.
`dmd.py`	Exact DMD via the SVD (rank truncation, eigenvalues, modes, amplitudes).
`observables.py`	Polynomial dictionary and a physics-informed trigonometric dictionary for the pendulum.
`edmd.py`	EDMD: ridge-regularised least squares in the lifted space and read-out of the state.
`deep_koopman.py`	Autoencoder with a linear latent Koopman matrix, multi-step training, spectral stabilisation.
`metrics.py`	Normalised error curves and the *valid prediction time*.
`viz.py`	All plots (consistent, colour-blind-safe style).

The scripts `experiments/pendulum.py` and `experiments/lorenz.py` run the full studies (`python main.py`); `tests/` contains the `pytest` suite.

A correction relative to the first version of the repository. The first version compared DMD and EDMD *rollouts* (started from x_0 only) with the *reconstruction* of the Deep Koopman autoencoder, i.e. decode(encode(x_t)) evaluated at every true state x_t. That reconstruction never uses the learned dynamics, so it shows how well an autoencoder compresses points of the attractor, not how well the model predicts. The statement that Deep Koopman "nearly overlaps" the true attractor was therefore not a prediction result. In this version every method is evaluated in the same way: from the initial state only, rolled forward with its own dynamics, on held-out data. The reconstruction error is still reported, but separately (Figure 5).

2 Theory

2.1 The Koopman operator

Consider a flow $\dot{x} = f(x)$ on $\mathbb{R}^d$ with time-t map φ^t, or a discrete map $x_{k+1} = F(x_k)$ (for us $F = \varphi^{\Delta t}$). For an *observable*, i.e. a scalar function g of the state, the Koopman operator acts by composition,

$$(\mathcal{K}g)(x) = g\big(F(x)\big). \tag{1}$$

$\mathcal{K}$ is *linear* even when F is nonlinear, because $\mathcal{K}(ag_1 + bg_2) = a\,\mathcal{K}g_1 + b\,\mathcal{K}g_2$; the price is that it acts on an infinite-dimensional space of functions [1, 6]. If $\mathcal{K}\psi_j = \lambda_j\psi_j$ (ψ_j an eigenfunction) and the state coordinates are expanded as $x = \sum_j v_j\psi_j(x)$ with *Koopman modes* v_j, then

$$x_k = \sum_j v_j\,\lambda_j^k\,\psi_j(x_0), \tag{2}$$

so the dynamics decouple into modes that grow or decay at rate $\log|\lambda_j|/\Delta t$ and oscillate at angular frequency $\arg\lambda_j/\Delta t$. The goal of the three methods below is to find a finite-dimensional approximation of $\mathcal{K}$ from data and to use it to predict.

2.2 Exact DMD

Given snapshot pairs $(x_k, y_k = F(x_k))$ arranged as columns of $X, Y \in \mathbb{R}^{d\times m}$, DMD finds the best linear operator $A = YX^{\dagger}$ and its spectrum [2, 3]. With the thin SVD $X = U\Sigma V^{*}$ truncated to rank r:

1. reduced operator $\tilde{A} = U^{*}YV\Sigma^{-1} \in \mathbb{R}^{r\times r}$;
2. eigendecomposition $\tilde{A}W = W\Lambda$;
3. *exact* DMD modes $\Phi = YV\Sigma^{-1}W$, which satisfy $A\Phi = \Phi\Lambda$;
4. amplitudes $b = \Phi^{\dagger}x_0$ and prediction $x_k = \mathrm{Re}\big(\Phi\Lambda^{k}b\big)$.

Seen from the Koopman viewpoint, DMD approximates $\mathcal{K}$ on the span of the state coordinates $\{x_1, \ldots, x_d\}$ only. This is exact if the dynamics are linear, but for a nonlinear system the span of the coordinates is not invariant under $\mathcal{K}$, which is the reason DMD struggles on both benchmarks. The continuous-time spectrum is $\omega_j = \log(\lambda_j)/\Delta t$; its real part is a growth rate and its imaginary part an angular frequency.

2.3 Extended DMD

EDMD enlarges the function space [4]. A *dictionary* of p observables $\psi(x) = [\psi_1(x), \ldots, \psi_p(x)]$ (a row vector) lifts the state, and $K \in \mathbb{R}^{p\times p}$ is fitted by ridge-regularised least squares over all pairs,

$$K = \arg\min_{K} \sum_{k} \|\psi(y_k) - \psi(x_k)K\|^2 + \lambda m \|K\|_F^2 . \tag{3}$$

For $g = \psi a$ one has $\mathcal{K}g \approx \psi\,(Ka)$. The state is read back through a linear read-out B with $x \approx \psi(x)B$ (fitted by least squares; because the first d features are the state itself, B simply selects them). A prediction from x_0 is $\hat{x}_k = \psi(x_0)K^{k}B$.

Feature standardisation is a diagonal similarity transform of K and leaves its spectrum unchanged; it is used only to keep the least-squares problem well conditioned. When the dictionary spans an *invariant subspace* of $\mathcal{K}$ the lifted dynamics are exactly linear. A classical example is the map $x_1^{+} = \mu x_1$, $x_2^{+} = \lambda x_2 + c\,x_1^2$, for which $\{x_1, x_2, x_1^2\}$ is invariant with eigenvalues μ, λ, μ^2; the test suite checks that EDMD recovers this system to machine precision while DMD cannot.

Physics-informed observables for the pendulum. For $\ddot{\theta} = -\omega_0^2 \sin\theta - c\dot{\theta}$ the dictionary uses the features

$$\theta,\ \ \dot{\theta},\ \ 1,\ \ \dot{\theta}^{\,p}\sin(k\theta),\ \ \dot{\theta}^{\,p}\cos(k\theta)\quad (p = 0, \ldots, P;\ k = 1, \ldots, N_h),\ \ \dot{\theta}^{\,p}\ (p = 2, \ldots, P). \tag{4}$$

The trigonometric terms respect the 2π-periodicity of the angle, and they are *closed under differentiation up to the truncation order*: $\frac{d}{dt}\sin\theta = \dot{\theta}\cos\theta$, $\frac{d}{dt}\cos\theta = -\dot{\theta}\sin\theta$ and $\frac{d}{dt}\dot{\theta} = -\omega_0^2\sin\theta - c\dot{\theta}$. Each differentiation raises the velocity degree by one, so no finite P gives an exactly invariant space; the number of harmonics N_h and the velocity degree P control how well it is approximated. For the Lorenz system, whose vector field is quadratic, the dictionary is the set of monomials up to a chosen degree.

2.4 Deep Koopman autoencoder

Instead of choosing observables by hand, an encoder $\phi : \mathbb{R}^d \to \mathbb{R}^L$ and a decoder $\psi : \mathbb{R}^L \to \mathbb{R}^d$ are learned jointly with a linear latent operator $K \in \mathbb{R}^{L\times L}$, so that $\phi(x_{k+1}) \approx K\phi(x_k)$ [5]. Training

uses windows $x_k, \dots, x_{k+M}$ and minimises

$$\mathcal{L}_{\text{rec}} = \big\|x_k - \psi(\phi(x_k))\big\|^2, \tag{5}$$

$$\mathcal{L}_{\text{lin}} = \tfrac{1}{M} \sum_{m=1}^{M} \big\|\phi(x_{k+m}) - K^m \phi(x_k)\big\|^2, \tag{6}$$

$$\mathcal{L}_{\text{pred}} = \tfrac{1}{M} \sum_{m=1}^{M} \big\|x_{k+m} - \psi\big(K^m \phi(x_k)\big)\big\|^2, \tag{7}$$

with total loss $\mathcal{L} = \mathcal{L}_{\text{rec}} + \mathcal{L}_{\text{lin}} + \mathcal{L}_{\text{pred}}$ (all in standardised state units). The reconstruction term alone would let the model memorise the data without learning any dynamics, the linearity term forces the latent space to evolve linearly, and the prediction term makes a rollout from a single initial state meaningful. Both networks are two-hidden-layer MLPs with tanh activations; K is initialised orthogonal and trained with Adam (learning rate decayed exponentially from $2 \cdot 10^{-3}$ to 10^{-4}) with gradient clipping.

A learned K can have an eigenvalue marginally outside the unit circle (e.g. 1.02). This is harmless over the short training windows but blows up in a long rollout. The method `DeepKoopman.stabilize` therefore rescales only the offending eigenvalues onto the unit circle, keeping the eigenvectors; the Lorenz experiment adopts it only if it lowers the *validation* error.

3 Implementation notes

ODE solvers. All data come from solvers written for this project. The classical Runge–Kutta step is

$$y_{n+1} = y_n + \tfrac{h}{6}\,(k_1 + 2k_2 + 2k_3 + k_4), \tag{8}$$

and the adaptive integrator is the Dormand–Prince embedded pair [7]: the difference of its fifth- and fourth-order solutions estimates the local error, and the step is updated as $h \leftarrow h \cdot \min\{10, \max\{0.2,\ 0.9\,\varepsilon^{-1/5}\}\}$ with ε the error scaled by $\text{atol} + \text{rtol}\,|y|$. Steps are clipped so that the solution lands *exactly* on the requested output grid, so no interpolation is needed. The tests verify that Euler and RK4 converge with orders 1 and 4, that the adaptive scheme meets its tolerance, that the Lorenz solution agrees with SciPy's DOP853, and that the undamped pendulum conserves its energy to 10^{-6}. Training data use RK4 with an internal step of at most 0.005.

Test suite. `pytest` runs unit tests of each module plus short end-to-end runs of both experiments and the command-line entry point. Notable checks: exact recovery of a linear operator and its spectrum by DMD; exact EDMD on a system with an invariant dictionary; absence of training windows that straddle two trajectories; reproducibility with a fixed seed; that Deep Koopman actually learns the dynamics of a linear system and generalises to unseen initial states; and that every figure is written to disk.

4 Experimental protocol

Pendulum. A damped pendulum with $g/L = 9.81\,\text{s}^{-2}$ and damping $c = 0.2\,\text{s}^{-1}$ is sampled every $\Delta t = 0.05\,\text{s}$ for 12 s. Initial conditions are drawn at random from a box ($|\theta_0| \le 2.7$, $|\dot\theta_0| \le 4$) restricted to the libration regime (energy below 95% of the separatrix). There are 60 training, 12 validation and 30 test trajectories. The small-angle period is $2\pi/\omega_0 \approx 2.0\,\text{s}$, so a rollout covers about six periods.

Lorenz. One long trajectory on the attractor with the classical parameters ($\sigma = 10$, $\rho = 28$, $\beta = 8/3$; the spin-up transient is discarded) is sampled every $\Delta t = 0.02$. It is split *chronologically* into 10000 training, 2000 validation and 4000 test steps, and the test segment provides 46 different initial states. Since the largest Lyapunov exponent is $\lambda_{\max} \approx 0.906$, errors grow like $e^{\lambda_{\max} t}$ and time is measured in *Lyapunov times* $1/\lambda_{\max} \approx 1.10$.

Evaluation. Every model predicts the *whole* trajectory from its initial state alone. We report

- the *normalised error* $e(t) = \|\hat{x}(t) - x(t)\| / \sigma$, with σ the RMS distance of the training states from their mean ($e = 1$ is as wrong as always predicting the mean state);
- the *mean rollout error*, the average of $e(t)$ over time and initial states;
- the *valid prediction time*, the first time $e(t)$ exceeds 0.2 (the median over initial states is reported);
- the *one-step RMSE*, the error of a single application of the learned map.

Model selection. All hyper-parameters are chosen on the *validation* data and the test data are used once at the end: the EDMD dictionary (harmonics $N_h \in \{1, \dots, 4\}$ and velocity degree $P \in \{1, \dots, 6\}$ for the pendulum; polynomial degree 2–4 for Lorenz), and for Deep Koopman the latent dimension and the prediction horizon M from a small grid (documented in the code comments of `experiments/`). DMD has no hyper-parameter besides its rank ($= d$).

5 Results

5.1 Pendulum

Table 1 summarises the held-out test trajectories. EDMD with the physics-informed dictionary (4 harmonics, velocity degree 5, 55 observables, chosen on validation) is by far the best: its one-step map is accurate to 10^{-6} and the median rollout stays below the 0.2 threshold for the whole 12 s horizon. The $\geq$ sign marks that cap: that number is a lower bound, not a measured crossing time, and the column "never above threshold" gives the fraction of test runs that really never crossed it. Deep Koopman (16-dimensional latent space, 60-step prediction loss, 4000 iterations) lies in between, while plain DMD, which can only represent a single fixed-frequency linear oscillation, loses track after about half a period.

Table 1: Pendulum, held-out initial conditions, rollout from the initial state alone. The $\geq$ marks a value capped by the 12 s horizon.

Method	mean error	valid time (s)	valid (periods)	never exceeded	one-step RMSE
DMD	0.624	1.10	0.5	0%	3.9×10^{-2}
EDMD	0.064	≥ 12.00	≥ 6.0	87%	1.4×10^{-6}
Deep Koopman	0.197	6.12	3.1	7%	2.5×10^{-2}

The mechanism is visible in the amplitude sweep (Figure 3): the pendulum's frequency drops as the amplitude grows, which no single linear model on (θ, ω) can reproduce, so the DMD error is large across the whole range and rises sharply for the largest amplitudes. The dictionary of EDMD contains $\sin k\theta$ and $\cos k\theta$ terms and expresses this nonlinearity as a linear map in the lifted space.

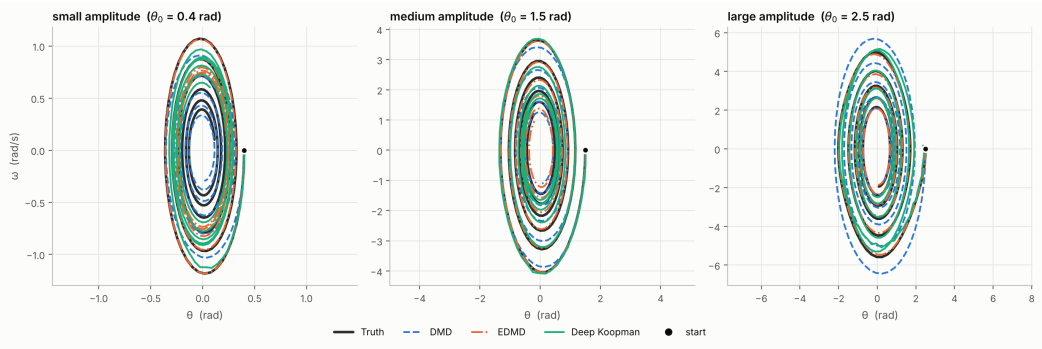


Figure 1: Phase portraits of three releases (small, medium and large amplitude).

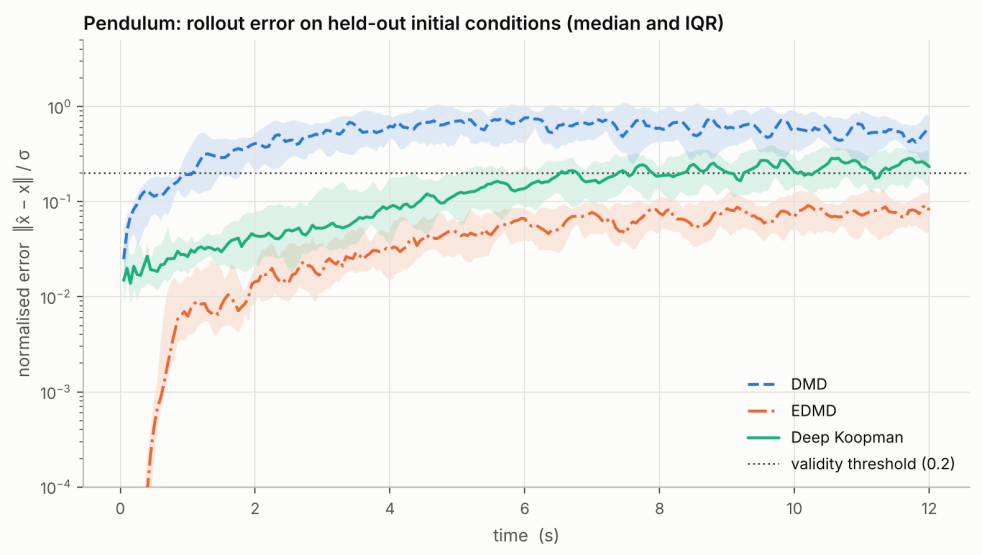


Figure 2: Rollout error versus time (median and inter-quartile range) on the test set.

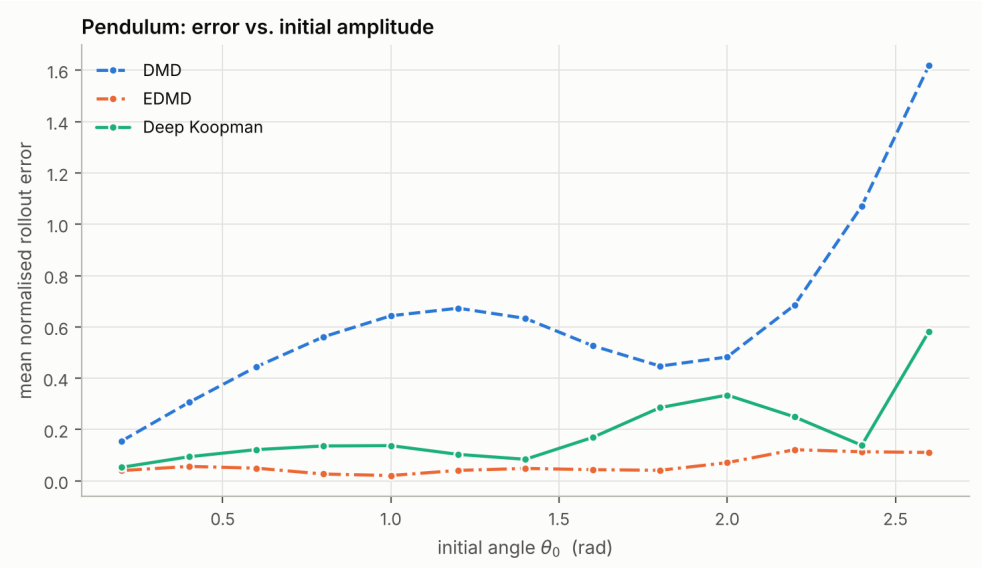


Figure 3: Mean rollout error versus initial amplitude.

5.2 Lorenz attractor

Lorenz is chaotic, so every method fails within a fraction of a Lyapunov time (Table 2); this is expected rather than a bug, since a finite-dimensional linear model of a chaotic flow cannot track a trajectory over long horizons. EDMD (degree 4 polynomials, 34 observables) has the best one-step accuracy and the best mean error, yet a short valid time (0.2 Lyapunov times). Deep Koopman has the longest valid time (0.44) but a large one-step RMSE, because a single step includes the autoencoder reconstruction error. Figure 5 separates the sources of error for Deep Koopman: the reconstruction alone is small, one step roughly triples it, and after one Lyapunov time the prediction error is ≈ 0.6.

Table 2: Lorenz, 46 test initial states. Times in Lyapunov times.

Method	mean error	median valid	mean valid	never exceeded	one-step RMSE
DMD	1.066	0.05	0.05	0%	1.0×10^{0}
EDMD	0.782	0.20	0.21	0%	1.1×10^{-3}
Deep Koopman	0.952	0.44	0.44	0%	7.8×10^{-1}

Table 3: Deep Koopman normalised error on Lorenz at increasing horizons.

Horizon	normalised error
reconstruction (no dynamics)	0.024
1 step ahead	0.072
1 Lyapunov time	0.627
3 Lyapunov times	1.194

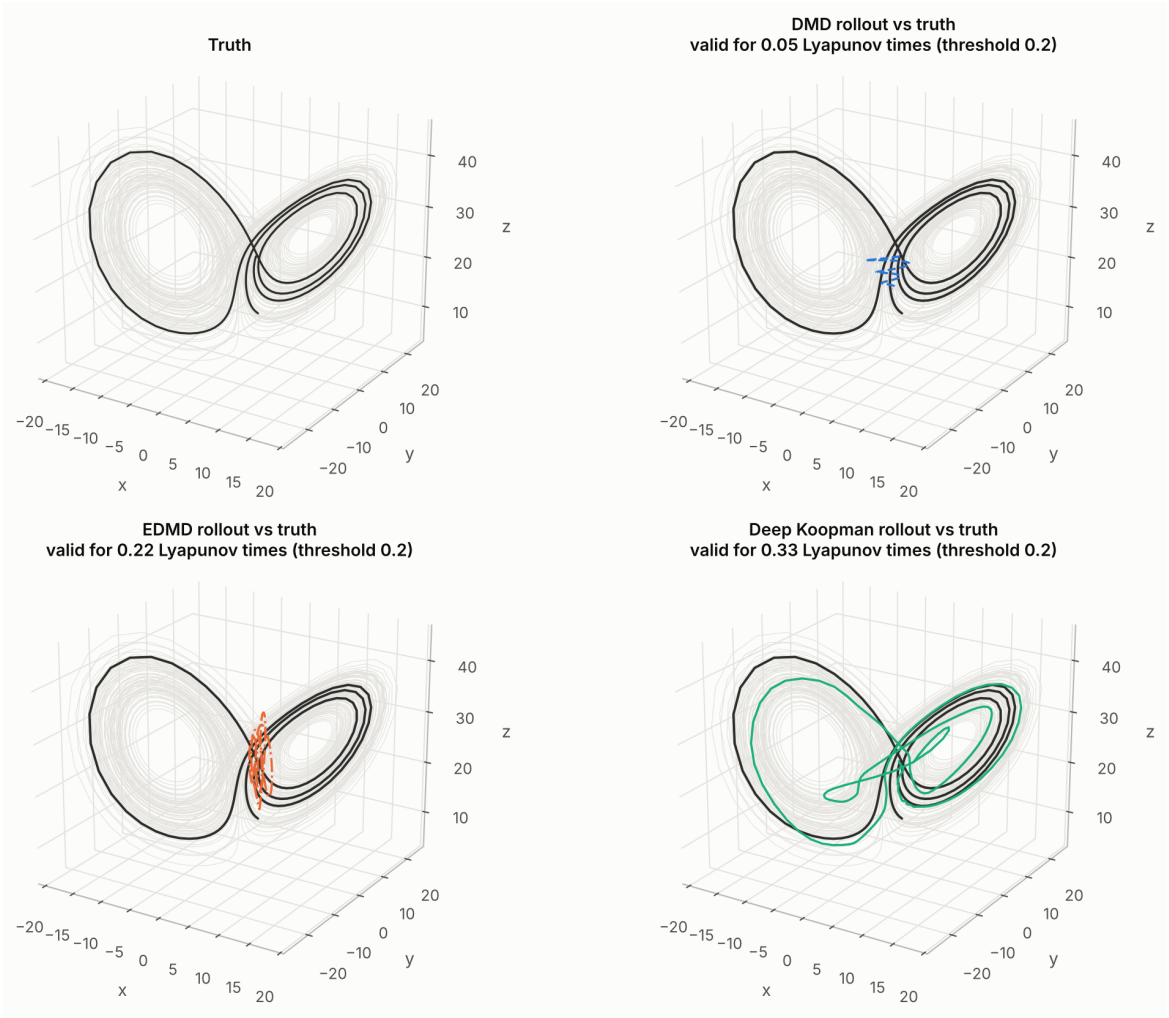


Figure 4: One test trajectory and each model's rollout from its initial state.

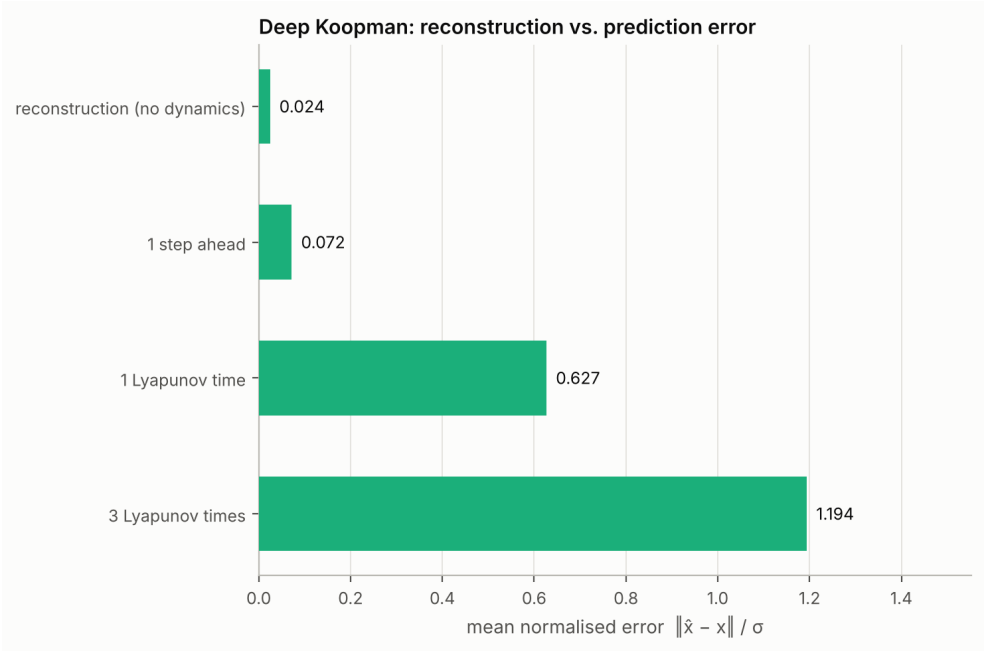


Figure 5: Deep Koopman error ladder: reconstruction, one step, one and three Lyapunov times.

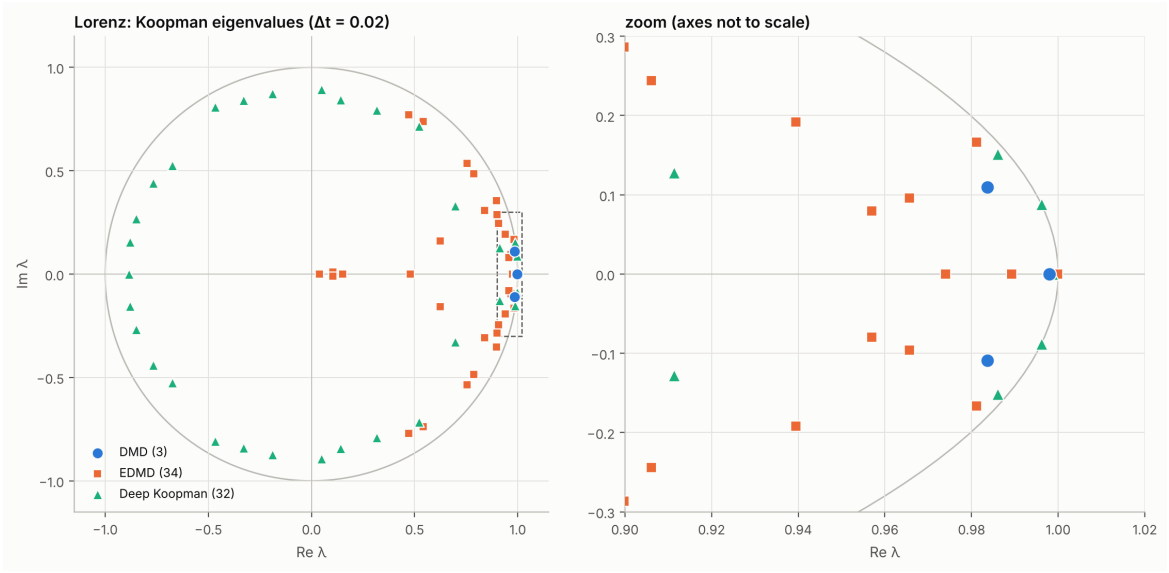


Figure 6: Eigenvalues of the learned operators (left: unit disc; right: magnified region near $\lambda = 1$).

5.3 Remaining figures

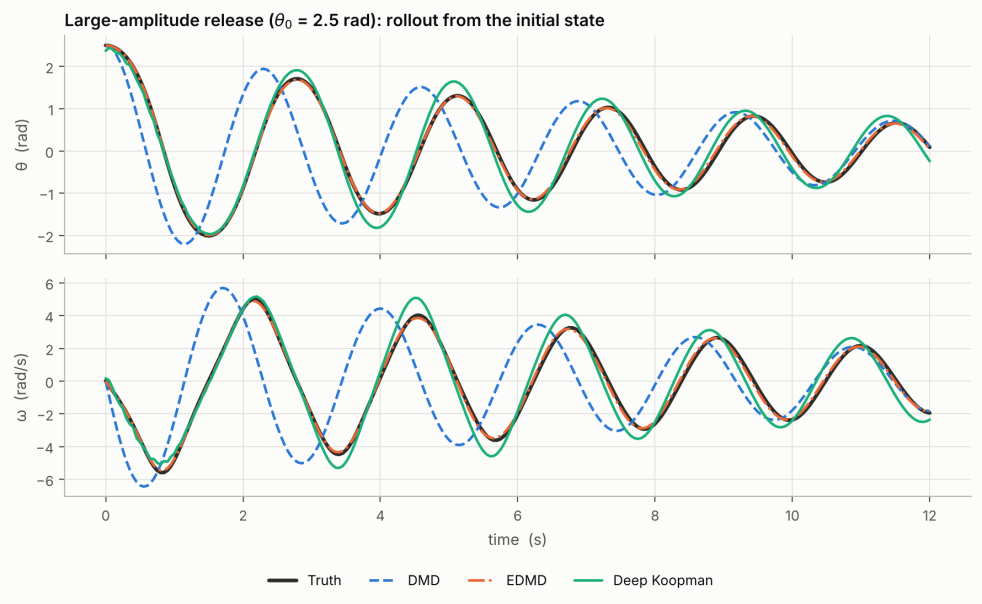


Figure 7: Time series of the large-amplitude release.

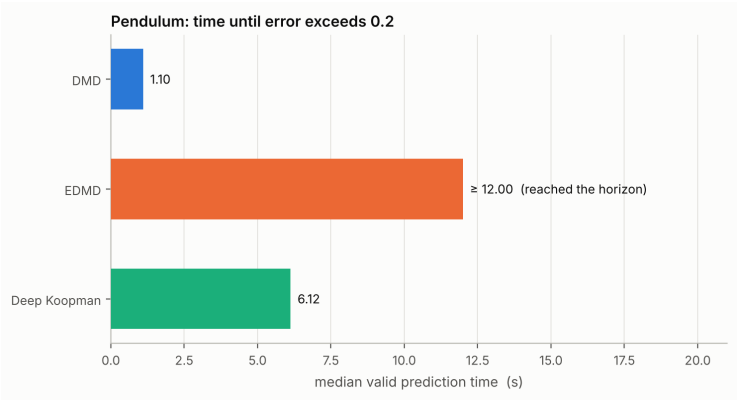


Figure 8: Pendulum valid prediction time (the EDMD bar reached the horizon).

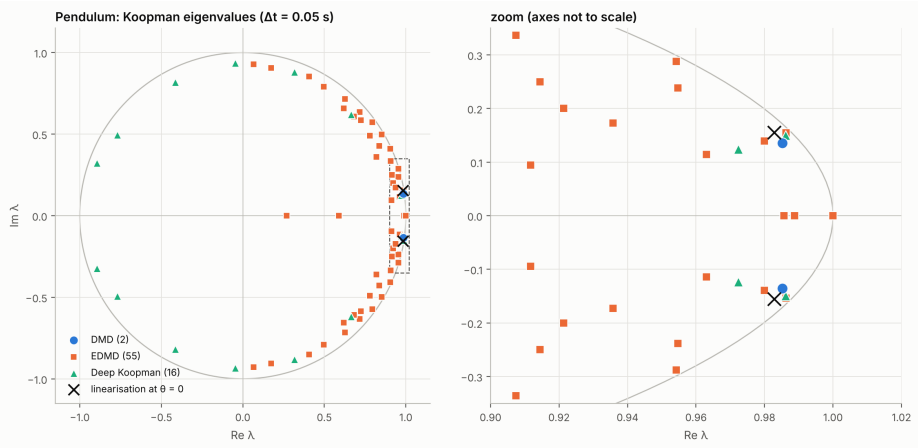


Figure 9: Pendulum operator spectra.

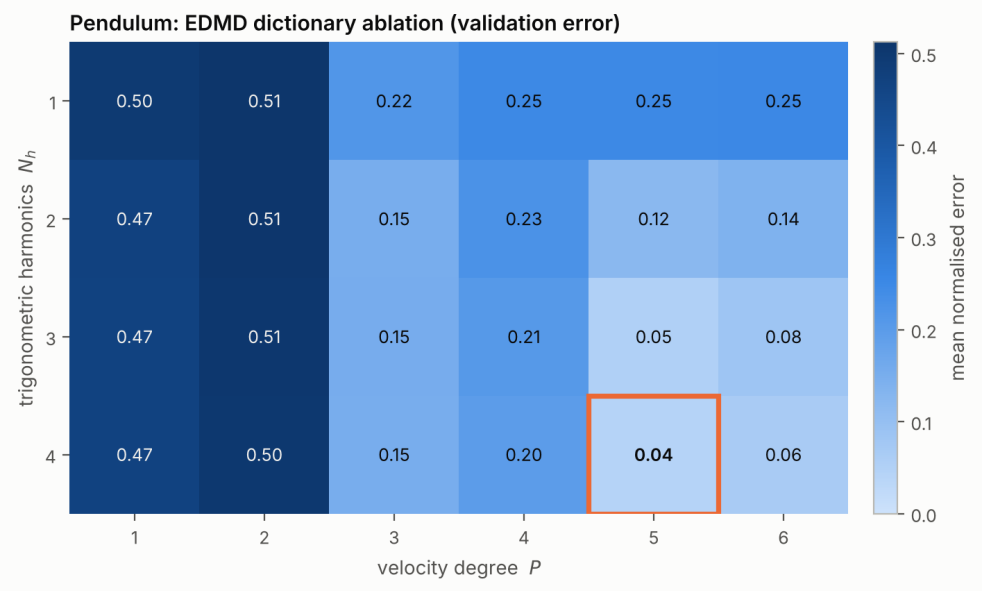


Figure 10: EDMD dictionary ablation on validation data.

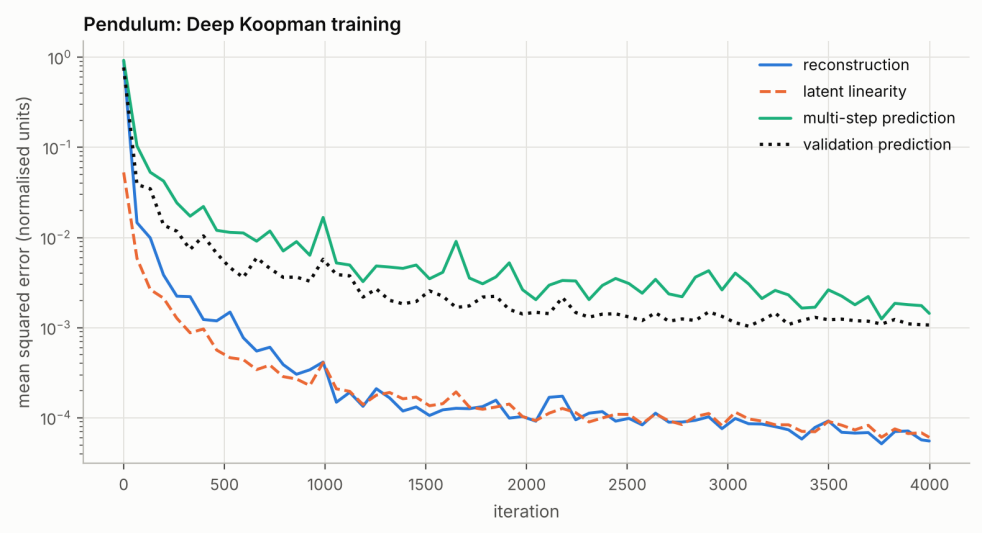


Figure 11: Deep Koopman training history (pendulum).

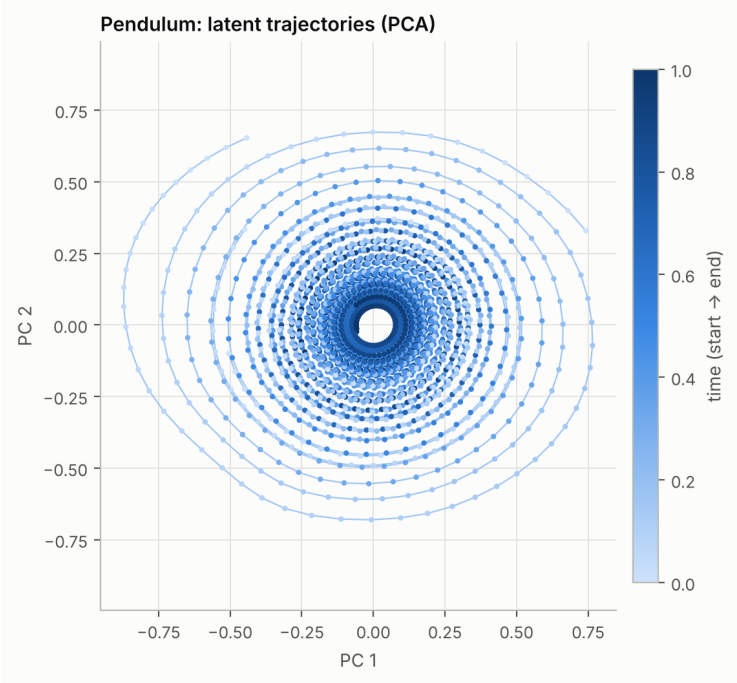


Figure 12: Pendulum latent trajectories (PCA).

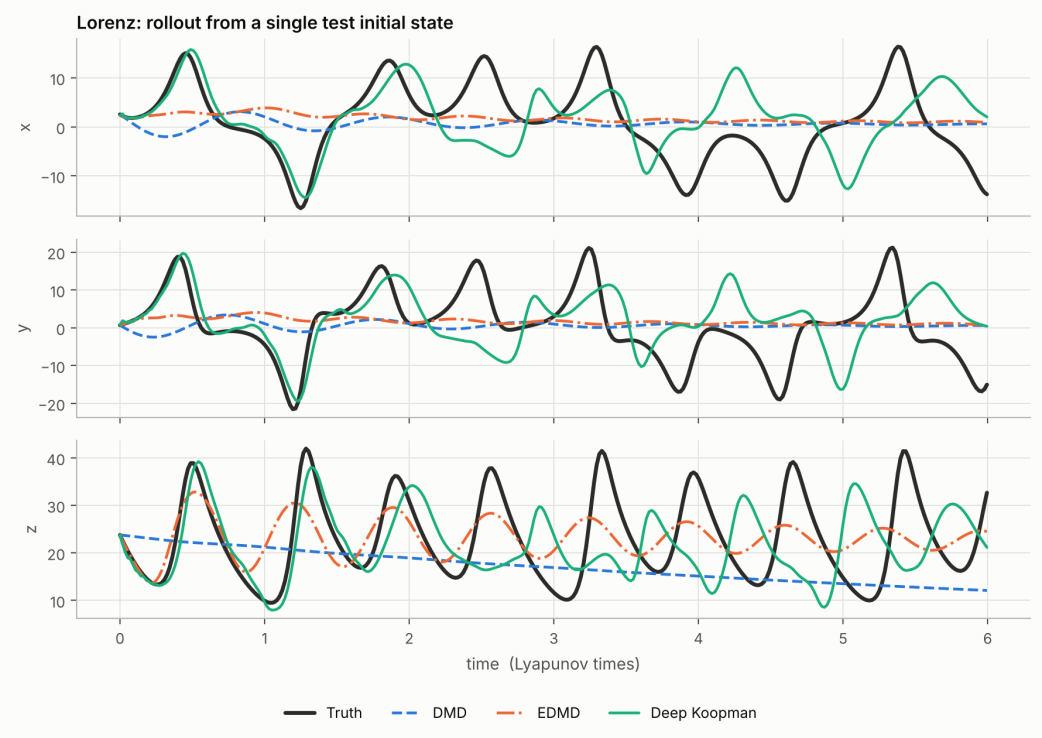


Figure 13: Lorenz time series of one test trajectory.

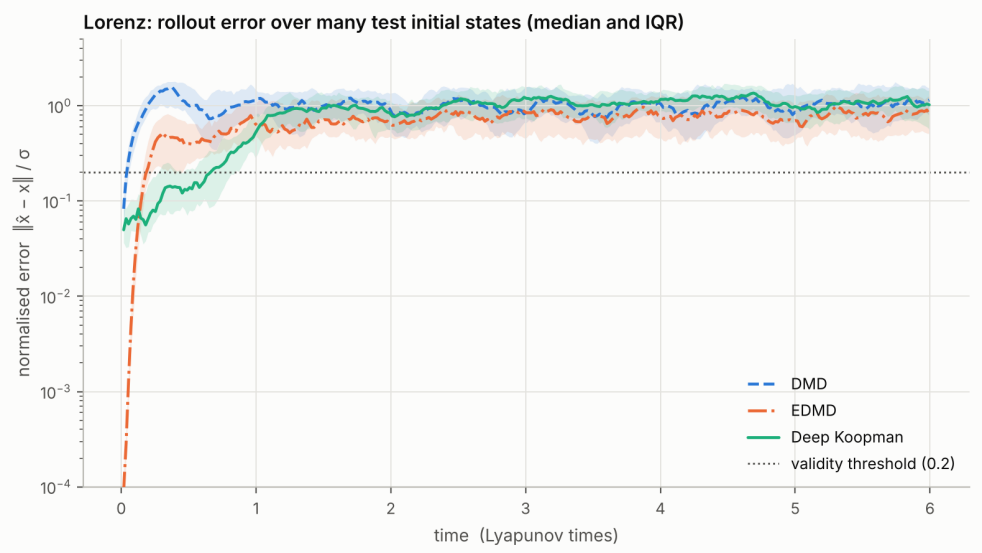


Figure 14: Lorenz rollout error versus time.

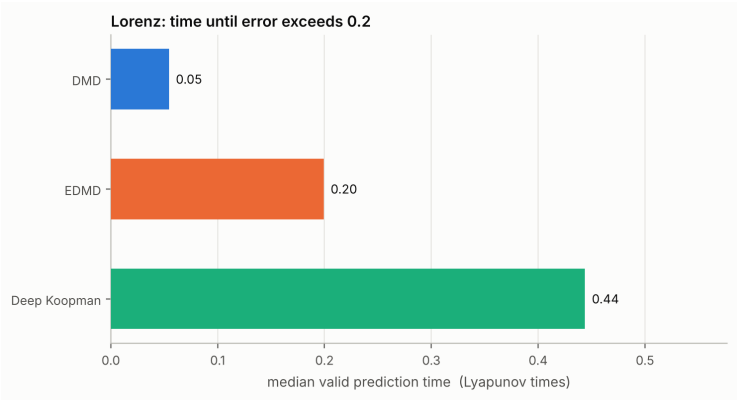


Figure 15: Lorenz valid prediction time.

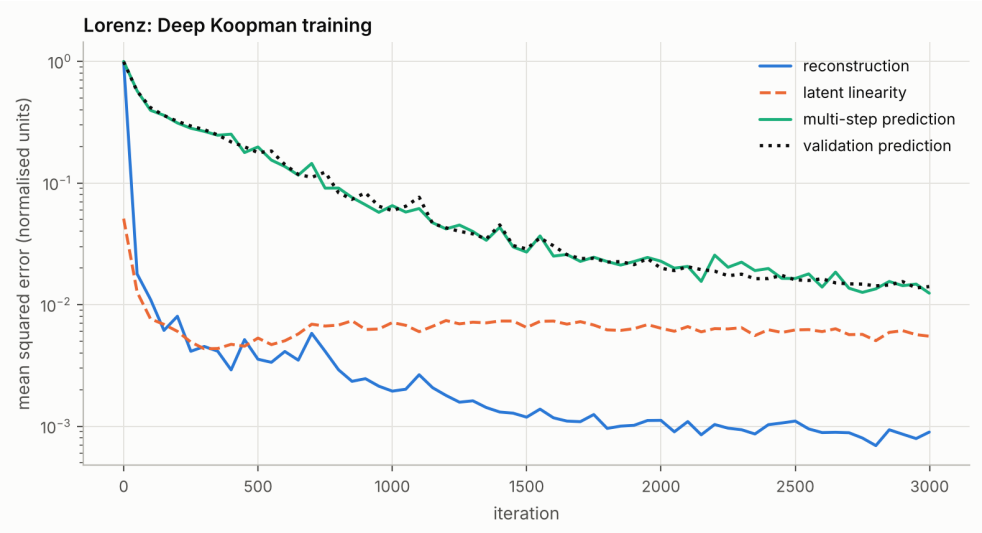


Figure 16: Deep Koopman training history (Lorenz).

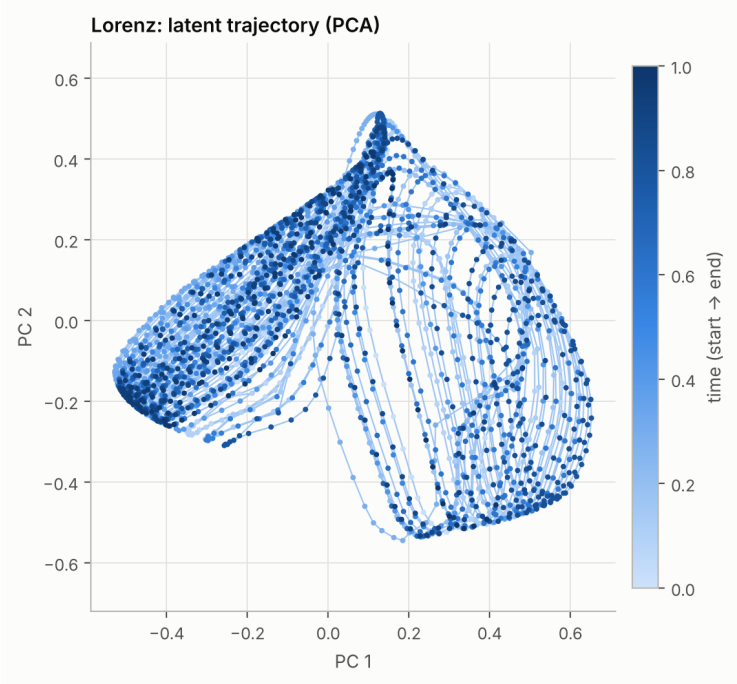


Figure 17: Lorenz latent trajectories (PCA).

6 How to read each figure

All figures are in `results/figures/`. Colours are colour-blind safe and every method also has its own line style.

Phase portraits. Each panel shows (θ, ω) for one release. The truth (dark) spirals into the origin because of damping; a model is good when its curve lies on the truth. DMD's curve decays at the wrong rate/frequency for large amplitude.

Time series. $\theta(t)$ and $\omega(t)$ for the large release. Look for phase drift: a model whose peaks gradually move away from the truth has the wrong frequency.

Error curves. Log-scale normalised error versus time; line = median, band = inter-quartile range, dashed line = the 0.2 validity threshold. The time where a curve crosses the dashed line is that method's valid prediction time.

Valid-time bars. Median valid time per method. A bar labelled "$\geq$ value" reached the horizon, so the number is a lower bound.

Error versus amplitude. Mean error per initial amplitude θ_0; a rising curve means the method degrades in the nonlinear regime.

Spectrum. Eigenvalues of the learned discrete operators in the complex plane; the circle is $|\lambda| = 1$. Points inside the circle decay, points on it oscillate indefinitely. The reference (linearised pendulum) is the target near $\lambda = 1$; the right panel magnifies that region and its axes are not to scale.

Training history. Deep Koopman loss terms (reconstruction, linearity, prediction) and validation prediction error versus iteration; all should decrease, and a growing validation curve signals overfitting.

Latent PCA. Latent trajectories projected on two principal components; for a pendulum these should be smooth spirals.

EDMD ablation. Validation error as a function of harmonics N_h and velocity degree P; the outlined cell is the choice.

Lorenz attractor comparison. Four 3D panels (truth, DMD, EDMD, Deep Koopman); the grey background is the attractor, the dark curve the truth over the plotted window, the coloured curve a rollout. A rollout that collapses to a point or small loop has lost the dynamics; the panel title gives the valid time.

Lorenz error ladder. Deep Koopman error for reconstruction, one step, one and three Lyapunov times: it shows whether the error comes from the autoencoder or from propagating the dynamics.

7 Limitations and honest caveats

- **Single seed.** All numbers come from one seed (0); differences between methods on Lorenz are small and might change with other seeds. No confidence intervals were computed.
- **Deep Koopman tuning.** Its hyper-parameters were chosen on a modest validation grid, not exhaustively; a larger search might improve it.
- **Lorenz is hard for any finite linear Koopman model.** Valid times below half a Lyapunov time are a fundamental limit of this approach, not only of this implementation. The Deep Koopman eigenvalues were slightly outside the unit circle (radius ≈ 1.018) and a post-hoc rescaling was used, selected on validation data only.
- **Capped pendulum EDMD result.** The EDMD median valid time is limited by the 12 s horizon, so the true value is larger.
- **Dictionary parity effect.** The EDMD validation error depends on whether the velocity degree P is odd or even (see the ablation figure); I observed it but did not investigate its cause.
- **One-step error of Deep Koopman** is dominated by the autoencoder's reconstruction error.

8 Reproducing the results

```
pip install -r requirements.txt
pytest                       # unit + smoke tests
python main.py               # both experiments -> results/
python main.py --quick       # a few-seconds smoke run
python report/make_tables.py
cd report && pdflatex koopman_report.tex      # rebuild this report
```

References

[1] B. O. Koopman, Hamiltonian systems and transformation in Hilbert space, *Proc. Natl. Acad. Sci. USA* **17**(5), 315–318 (1931).

[2] P. J. Schmid, Dynamic mode decomposition of numerical and experimental data, *J. Fluid Mech.* **656**, 5–28 (2010).

[3] J. H. Tu, C. W. Rowley, D. M. Luchtenburg, S. L. Brunton, J. N. Kutz, On dynamic mode decomposition: theory and applications, *J. Comput. Dyn.* **1**(2), 391–421 (2014).

[4] M. O. Williams, I. G. Kevrekidis, C. W. Rowley, A data-driven approximation of the Koopman operator: extending dynamic mode decomposition, *J. Nonlinear Sci.* **25**(6), 1307–1346 (2015).

[5] B. Lusch, J. N. Kutz, S. L. Brunton, Deep learning for universal linear embeddings of nonlinear dynamics, *Nat. Commun.* **9**, 4950 (2018).

[6] S. L. Brunton, M. Budišić, E. Kaiser, J. N. Kutz, Modern Koopman theory for dynamical systems, *SIAM Rev.* **64**(2), 229–340 (2022).

[7] J. R. Dormand, P. J. Prince, A family of embedded Runge–Kutta formulae, *J. Comput. Appl. Math.* **6**(1), 19–26 (1980).

[8] E. N. Lorenz, Deterministic nonperiodic flow, *J. Atmos. Sci.* **20**(2), 130–141 (1963).